
ELITEALPHA: Illuminating the Alpha Factor Space with MAP-Elites

Ruili Liu* Yiwei Jiang*

2025316149 2023000000

Department of Computer Science and Technology, Tsinghua University
liu-rl25@mails.tsinghua.edu.cn xxx@mails.tsinghua.edu.cn

1 Introduction

Financial markets are high-dimensional, non-stationary stochastic systems whose returns exhibit heavy tails [5], time-varying volatility [4], and strong cross-sectional dependence [13]. Under such low signal-to-noise conditions, quantitative investing relies on *alpha mining*, the search for formulaic signals that map raw market observations to predictors of forward returns. Classical pipelines depend on human experts or symbolic search methods such as genetic programming [7] and reinforcement learning [20], which explore a fixed operator library but often produce factors that are either overfit to historical statistics or economically opaque.

Driven by advances in code generation and domain reasoning, LLM-based alpha mining has matured rapidly. Recent agentic frameworks simulate the workflow of a quantitative researcher by iteratively proposing hypotheses, synthesizing executable factor expressions, and refining them through backtesting feedback. Representative systems include AlphaAgent [17], which regularizes generation against factor crowding via AST-based novelty penalties; QuantaAlpha [6], which performs trajectory-level mutation and crossover with complexity and redundancy constraints; and AlphaHunter [15], which couples LLMs with Monte Carlo Tree Search over the expression space. These methods substantially reduce the trial-and-error cost of factor research while preserving interpretability.

Despite this progress, a structural limitation remains: *factor crowding*. Because LLMs are pretrained on the same public corpora and conditioned on noisy backtest rewards, iterative refinement tends to over-exploit a small neighborhood of well-known signals such as momentum, RSI, or size, leading to redundant alphas and accelerated alpha decay [17]. Existing remedies treat diversity as a soft constraint: AlphaAgent penalizes AST similarity, while QuantaAlpha bounds expression complexity. Both approaches discourage similarity *after* a candidate is proposed, but they do not guarantee systematic coverage of the hypothesis space, and the resulting factor pool is still biased toward whichever niche the search trajectory first locks onto.

To this end, we propose ELITEALPHA, an evolutionary alpha mining framework built on MAP-Elites [11], a Quality-Diversity (QD) algorithm that maintains an archive of elite solutions across a discretized behavioral feature space. ELITEALPHA discretizes the factor space into a two-dimensional grid whose axes are economically meaningful behavior descriptors: a categorical axis indexing factor families such as momentum, reversal, volatility, volume-price, and cross-sectional signals, and an ordinal axis indexing structural complexity measured by the depth of the abstract syntax tree (AST) of the factor expression. At each iteration, a parent factor is sampled from the archive, an LLM acts as a mutation or crossover operator to generate a child factor, and the child is admitted into its corresponding cell only if the cell is empty or if the child achieves a higher information coefficient than the current incumbent. Because each grid cell retains only its local elite, every behavioral niche is occupied by a high-quality yet structurally distinct factor by construction, simultaneously addressing quality and diversity.

*Equal contributions.

The combination of MAP-Elites with LLM operators has been validated in adjacent domains, including QDAIF for open-ended text generation [1], CycleQD for multi-skill LLM training [8], and DeepMind’s AlphaEvolve program database for code discovery [12]. Building on these insights, we bring Quality-Diversity search to formulaic alpha mining for the first time, leveraging a behavior space whose axes admit a particularly natural and interpretable design grounded in financial semantics. Our contributions are summarized as follows:

- We formulate alpha factor mining as a Quality-Diversity optimization problem and introduce ELITEALPHA, the first MAP-Elites enhanced framework for formulaic alpha discovery, with behavior descriptors that align with established factor taxonomies.
- We plan to evaluate ELITEALPHA on CSI 300 and CSI 500 within the Qlib framework [19], comparing against AlphaAgent [17], QuantAlpha [6] and AlphaEvolve [12] on both predictive metrics (IC, Rank IC, QD-Score) and portfolio metrics (annualized return, information ratio, factor crowding).

2 Related Work

LLM Agents for Alpha Mining. The introduction of large language models into quantitative research has rapidly reshaped how alpha factors are discovered. Early efforts such as Alpha-GPT [18] framed factor mining as a human-AI interactive loop, using retrieval-augmented prompting over financial literature to surface candidate signals. Subsequent systems formalized this loop into multi-agent pipelines that mimic the workflow of a quantitative researcher: hypothesis generation, executable factor construction, and backtest-driven refinement. AlphaAgent [17] instantiates this paradigm with three cooperating agents and combats factor crowding through AST similarity penalties, hypothesis-factor consistency checks, and explicit complexity bounds. RD-Agent [9] extends the scope of automation to joint factor and model co-optimization, scheduling effort across research and development stages with a multi-armed bandit. QuantaAlpha [6] elevates the unit of evolution from individual factors to entire mining trajectories, performing trajectory-level mutation and crossover with semantic consistency constraints across hypothesis, expression, and code. Beyond closed-loop refinement, recent work explores alternative search strategies. AlphaHunter [15] couples LLMs with Monte Carlo Tree Search and a frequent subtree avoidance mechanism that explicitly steers generation away from over-represented patterns. FactorMAD [3] adopts a multi-agent debate architecture in which two agents with distinct priors iteratively critique and revise candidate factors, while CogAlpha [10] organizes mining as a seven-level cognitive hierarchy spanning macro market reasoning down to candlestick geometry. Despite these advances, diversity in nearly all of these systems is treated as a soft penalty applied after generation, leaving the resulting factor pools vulnerable to homogenization and accelerated alpha decay.

Quality-Diversity Optimization with LLM Operators. QD algorithms [14] depart from single-objective optimization by simultaneously seeking many high-performing solutions that differ along user-specified behavioral dimensions. MAP-Elites [11] is the canonical instance of this paradigm, maintaining an archive of elite solutions across a discretized behavior space and updating each cell through elitist replacement. Originally developed for robot morphology and control [2], MAP-Elites has since been extended to a wide range of domains including neural architecture search and procedural content generation. The combination of QD search with LLMs as variation operators is a more recent development. QDAIF [1] uses an LLM to both mutate text artifacts and judge their behavioral characteristics, achieving open-ended diversity in creative writing. CycleQD [8] cycles MAP-Elites across multiple quality and behavior axes to train LLMs that acquire complementary skills, while AlphaEvolve [12] employs a MAP-Elites-inspired program database to maintain diverse code candidates during automated algorithm discovery. These results show that LLMs are effective stochastic operators inside QD loops and that explicit niching scales naturally to symbolic and code-based artifacts. Our work brings this line of research into quantitative finance: by treating factor category and AST complexity as behavior descriptors, we obtain an interpretable, financially grounded behavior space in which LLM-driven mutation and crossover are forced to populate every niche, replacing post-hoc similarity penalties with constructive diversity guarantees.

3 Approach

We now describe the design of ELITEALPHA in detail. Section 3.1 formulates alpha factor mining as a Quality-Diversity optimization problem. Section 3.2 introduces the two-dimensional behavior space. Section 3.3 presents the LLM-driven variation operators. Section 3.4 gives the full algorithm. Section 3.5 extends the framework to an island-based parallel setting. Section 3.6 outlines our remaining work plan.

3.1 Problem Formulation

Let $\mathcal{F}$ denote the space of all syntactically valid factor expressions that can be constructed from a predefined operator library (e.g., arithmetic operators, rolling windows, ranking functions) and a set of raw market features (open, high, low, close, volume). Each factor $f \in \mathcal{F}$ maps a window of historical market data to a cross-sectional score vector. We evaluate the predictive quality of f by its information coefficient (IC), defined as the Spearman rank correlation between the factor scores and the forward returns over a held-out period:

$$\text{IC}(f) = \text{SpearmanCorr}(\mathbf{s}_f, \mathbf{r}^{\text{fwd}}), \quad (1)$$

where $\mathbf{s}_f$ is the cross-sectional score vector produced by f and $\mathbf{r}^{\text{fwd}}$ is the vector of next-period stock returns.

In a standard single-objective formulation, one seeks $f^* = \arg \max_{f \in \mathcal{F}} \text{IC}(f)$. However, a single high-IC factor is of limited practical value: portfolio construction requires a *pool* of factors that are both predictive and mutually complementary. We therefore cast the problem as Quality-Diversity optimization. Let $\mathbf{b}(f) \in \mathcal{B}$ be a behavior descriptor that characterizes the economic role and structural properties of f , where $\mathcal{B}$ is a discretized behavior space. The goal is to find, for every occupied cell $c \in \mathcal{B}$, an elite factor f_c^* such that

$$f_c^* = \arg \max_{f: \mathbf{b}(f)=c} \text{IC}(f). \quad (2)$$

The aggregate objective is the QD-Score, defined as the sum of IC values across all occupied cells:

$$\text{QD-Score} = \sum_{c \in \mathcal{B}} \text{IC}(f_c^*). \quad (3)$$

Maximizing QD-Score simultaneously encourages high per-cell quality and broad coverage of the behavior space, yielding a diverse and high-performing factor pool by construction.

3.2 Behavior Space Design

A key design choice in any MAP-Elites system is the definition of behavior descriptors. We define a two-dimensional behavior space $\mathcal{B} = \mathcal{C} \times \mathcal{D}$, where each axis captures an economically meaningful property of the factor.

Factor category axis $\mathcal{C}$. The horizontal axis is a categorical dimension that indexes the economic family to which a factor belongs. We adopt five categories aligned with standard factor taxonomies in quantitative finance: *momentum*, *reversal*, *volatility*, *volume-price*, and *cross-sectional*. Given a candidate factor expression, we prompt the LLM to classify it into one of these categories based on the operators and market features it uses. For instance, a factor that computes a rolling return difference is classified as momentum, while one that normalizes volume by its moving average falls under volume-price.

Complexity axis $\mathcal{D}$. The vertical axis is an ordinal dimension that measures the structural complexity of the factor expression. We define complexity as the depth of the abstract syntax tree (AST) of the expression after parsing. AST depth is a natural proxy for factor complexity: depth-1 expressions correspond to simple univariate transforms (e.g., $\text{Rank}(\text{close})$), while deeper trees combine multiple nested operations. We discretize this axis into L levels (e.g., $L = 5$, corresponding to AST depths 1 through 5).

The resulting behavior space is a $|\mathcal{C}| \times L$ grid. Each cell (c, d) represents a specific combination of economic category and structural complexity. Not all cells are guaranteed to be filled: certain category-complexity combinations may be infeasible or may not be discovered during the search. This is a well-known property of MAP-Elites [11] and does not affect the correctness of the algorithm.

```

1: procedure ELITEALPHA
2:    $\mathcal{P} \leftarrow \emptyset, \mathcal{X} \leftarrow \emptyset$  ▷ Initialize empty archive
3:   for iter = 1 to  $I$  do
4:     if iter  $\leq G$  then ▷ Initialization phase
5:        $f' \leftarrow \text{LLM\_INIT}(\text{seed\_category}[\text{iter}])$ 
6:     else ▷ Variation phase
7:        $f_1 \leftarrow \text{RANDOM\_SELECT}(\mathcal{X})$ 
8:       Draw  $u \sim \text{Uniform}(0, 1)$ 
9:       if  $u < p_{\text{cross}}$  and  $|\mathcal{X}| \geq 2$  then
10:         $f_2 \leftarrow \text{RANDOM\_SELECT}(\mathcal{X} \setminus \{f_1\})$ 
11:         $f' \leftarrow \text{LLM\_CROSSOVER}(f_1, f_2)$ 
12:       else
13:         $f' \leftarrow \text{LLM\_MUTATE}(f_1)$ 
14:        $c' \leftarrow \text{LLM\_CLASSIFY}(f')$  ▷ Determine factor category
15:        $d' \leftarrow \text{AST\_DEPTH}(f')$  ▷ Compute expression complexity
16:        $\mathbf{b}' \leftarrow (c', d')$  ▷ Behavior descriptor
17:        $p' \leftarrow \text{IC}(f')$  ▷ Evaluate on validation data
18:       if  $\mathcal{P}(\mathbf{b}') = \emptyset$  or  $\mathcal{P}(\mathbf{b}') < p'$  then
19:          $\mathcal{P}(\mathbf{b}') \leftarrow p'; \mathcal{X}(\mathbf{b}') \leftarrow f'$ 
20:   return  $(\mathcal{P}, \mathcal{X})$ 

```

Figure 1: The ELITEALPHA algorithm. G : number of initialization samples; I : total iterations; p_{cross} : crossover probability.

3.3 LLM-Driven Variation Operators

In classical MAP-Elites, variation is performed by domain-specific mutation and crossover operators defined over fixed-length genomes. In ELITEALPHA, factor expressions are variable-length symbolic programs, making hand-crafted genetic operators brittle. Instead, we use an LLM as a general-purpose variation operator, following the paradigm validated by QDAIF [1] and AlphaEvolve [12].

Mutation. Given a single parent factor f , the LLM is prompted to generate a modified version f' that preserves the core economic intuition but differs in operator choice, window length, or structural arrangement. The prompt includes the parent expression, its category and IC, and an instruction to explore a specific direction (e.g., “increase complexity by adding a normalization layer” or “replace the momentum signal with a volatility component”).

Crossover. Given two parent factors f_1 and f_2 sampled from different cells, the LLM is prompted to synthesize a child f' that combines elements of both parents. For example, one parent may supply the core signal logic while the other contributes a decay or normalization scheme. The prompt provides both parent expressions, their respective categories and IC values, and asks the LLM to produce a factor that integrates complementary aspects.

In both cases, the LLM output is parsed into an executable Python expression and validated against the operator library. Invalid or non-executable expressions are discarded, and a new variation is attempted. We use DeepSeek-R1 or Qwen as the backbone LLM, as both support long-context code generation and are freely accessible.

3.4 Algorithm

Algorithm 1 presents the complete ELITEALPHA procedure. The archive $\mathcal{X}$ stores elite factor expressions and $\mathcal{P}$ stores their IC values, both indexed by behavior descriptor. The algorithm begins with an initialization phase in which G random factor expressions are generated via LLM prompting with diverse seed instructions spanning all target categories. After initialization, each iteration samples a parent from the archive, applies mutation or crossover to produce a child, evaluates the child’s IC on the validation set, determines its behavior descriptor, and attempts elitist replacement in the corresponding cell. The category label is obtained by querying the LLM, and the AST depth is computed by parsing the expression.

3.5 Island-Based Parallel Extension

A single MAP-Elites archive can suffer from limited exploration when the LLM repeatedly conditions on a narrow subset of elites. To mitigate this, we adopt an island model [16] that runs K independent ELITEALPHA instances in parallel, each maintaining its own archive $(\mathcal{P}_k, \mathcal{X}_k)$ for $k = 1, \dots, K$. Each island targets a different subset of seed categories, encouraging divergent search trajectories.

Every M iterations, a migration step is performed. Each island selects a small number of its elite factors (e.g., one per occupied category) and broadcasts them to all other islands. Upon receiving a migrant factor f , an island computes its behavior descriptor and attempts elitist replacement in its own archive, following the same admission rule as in Algorithm 1. This mechanism allows high-quality factors discovered on one island to propagate across the population while preserving each island’s local diversity.

After all islands terminate, we merge the K archives by applying cell-wise elitist selection: for each behavior descriptor $\mathbf{b}$, the globally best factor across all islands is retained. The merged archive constitutes the final output of the system.

The island model offers two practical benefits. First, it enables straightforward parallelization across multiple GPUs or API endpoints, reducing wall-clock time roughly linearly in K . Second, it reduces the risk of premature convergence: even if one island collapses to a narrow region of the factor space, other islands continue to explore complementary niches, and migration periodically re-introduces diversity.

3.6 Work Plan

We have completed the literature review and the prototype design described above. Table 1 summarizes our remaining work.

Week	Tasks
7–8	Set up AlphaAgent [17] and QuantaAlpha [6] environments; study existing framework code and reproduce baseline results.
9–12	Implement the full MAP-Elites factor mining pipeline; implement island-based parallel extension; integrate LLM variation operators.
12–13	Factor evaluation (IC, ICIR, Rank IC); MAP-Elites loop debugging; comparative experiments against baselines; ablation studies on behavior space design and migration frequency; CSI 300/500 backtesting;
14–15	Write final report in NeurIPS format; prepare presentation slides;

Table 1: Remaining work plan.

References

- [1] Herbie Bradley, Andrew Dai, Hannah Teufel, Jenny Zhang, Koen Oostermeijer, Marco Bellagente, Jeff Clune, Kenneth Stanley, Grégory Schott, and Joel Lehman. Quality-diversity through AI feedback. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015. doi: 10.1038/nature14422.
- [3] Yitong Duan, Chuheng Zhang, and Jian Li. FactorMAD: A multi-agent debate framework based on large language models for interpretable stock alpha factor mining. In *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF)*, 2025. doi: 10.1145/3768292.3770377.
- [4] Robert F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4):987–1007, 1982.
- [5] Eugene F. Fama. The behavior of stock-market prices. *The Journal of Business*, 38(1):34–105, 1965.

- [6] Jun Han, Shuo Zhang, Wei Li, Zhi Yang, Yifan Dong, Tu Hu, Jialuo Yuan, Xiaomin Yu, Yumo Zhu, Fangqi Lou, et al. QuantaAlpha: An evolutionary framework for LLM-driven alpha mining. *arXiv preprint arXiv:2602.07085*, 2026.
- [7] Zura Kakushadze. 101 formulaic alphas. *Wilmott*, 2016(84):72–81, 2016.
- [8] So Kuroki, Taishi Nakamura, Takuya Akiba, and Yujin Tang. Agent skill acquisition for large language models via CycleQD. In *International Conference on Learning Representations (ICLR)*, 2025.
- [9] Yuante Li, Xu Yang, Xiao Yang, Minrui Xu, Xisen Wang, Weiqing Liu, and Jiang Bian. R&D-Agent-Quant: A multi-agent framework for data-centric factors and model joint optimization. *arXiv preprint arXiv:2505.15155*, 2025. To appear in NeurIPS 2025.
- [10] Fengyuan Liu et al. Cognitive alpha mining via LLM-driven code-based evolution. *arXiv preprint arXiv:2511.18850*, 2025.
- [11] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [12] Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [13] M. Hashem Pesaran. General diagnostic tests for cross-sectional dependence in panels. *Empirical Economics*, 60(1):13–50, 2021.
- [14] Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016. doi: 10.3389/frobt.2016.00040.
- [15] Yu Shi, Yitong Duan, and Jian Li. Navigating the alpha jungle: An LLM-powered MCTS framework for formulaic factor mining. *arXiv preprint arXiv:2505.11122*, 2025.
- [16] Zbigniew Skolicki and Kenneth A. De Jong. The influence of migration sizes and intervals on island models. In *Proceedings of the 2005 Conference on Genetic and Evolutionary Computation (GECCO)*, pages 1295–1302, Washington, DC, USA, 2005. ACM.
- [17] Ziyi Tang, Zechuan Chen, Jiarui Yang, Jiayao Mai, Yongsun Zheng, Keze Wang, Jinrui Chen, and Liang Lin. AlphaAgent: LLM-driven alpha mining with regularized exploration to counteract alpha decay. *arXiv preprint arXiv:2502.16789*, 2025.
- [18] Saizhuo Wang, Hang Yuan, Leon Zhou, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. Alpha-GPT: Human-AI interactive alpha mining for quantitative investment. *arXiv preprint arXiv:2308.00016*, 2023.
- [19] Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. Qlib: An AI-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189*, 2020.
- [20] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. Generating synergistic formulaic alpha collections via reinforcement learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5476–5486, 2023.